

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
OFFICE FOR INTERNATIONAL STUDY PROGRAM**

SEMESTER: 261

◇ . . . ◇



LAB 2 REPORT

INSTRUCTOR:

MR. PHAN VAN SY

Student Name: Du Duc Minh

Student ID: 2452734

Class ID: CC03

Ho Chi Minh City, October 04, 2026

Contents

1	Exercise 1	3
2	Exercise 2	5
3	Exercise 3	7
4	Exercise 4	8
5	Exercise 5	9
6	Exercise 6	10
7	Exercise 7 and Exercise 8	11
8	Exercise 9 and Exercise 10	12

1 Exercise 1

Schematic from Proteus for Exercise 1:

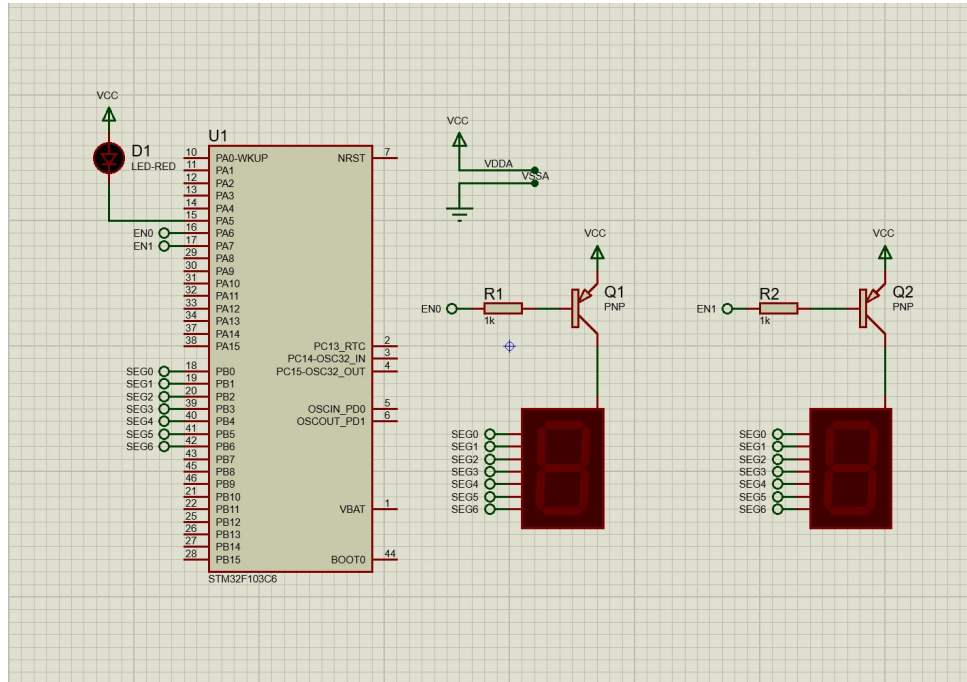


Figure 1: Proteus schematic for Exercise 1.

Source code for Ex1:

```
1 int ex1 = 1 ;
2 void run_ex1() {
3     display7SEG(ex1) ;
4     if(ex1 == 1) ex1 = 2 ;
5     else if(ex1 == 2) ex1 = 1 ;
6
7     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
8     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
9     if(ex1 == 2) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
10         GPIO_PIN_RESET);
11     else HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
12         GPIO_PIN_RESET);
13 }
```

Code inside HAL-TIM-PeriodElapsedCallback function:

```
1  int clk = 0 ;
2  void HAL_TIM_PeriodElapsedCallback ( TIM_HandleTypeDef * htim )
3  {
4      clk++ ;
5      timer_run() ; clk = 0 ;
6      if(clk == 51){ run_ex1() ; clk = 1 ;}
7      // if(clk == 26){ run_ex2() ; clk = 1 ;}
8      // if(clk == 26){ update7SEG(index++); if(index >= MAX_LED)
9          index = 0 ; clk = 1 ;} // for ex 3, 4
10     // if(clk == 26){ run_ex5() ; update7SEG(index++); if(index
        >= MAX_LED) index = 0 ;  clk = 1 ;} // for ex 5
11 }
```

What is the frequency of the scanning process?

One complete scanning cycle takes 1 second, since the switching time between two 7-segment LEDs is 0.5 seconds. Therefore, resulting in a scanning frequency of 1 Hz.

Schematic from Proteus for Exercise 2:

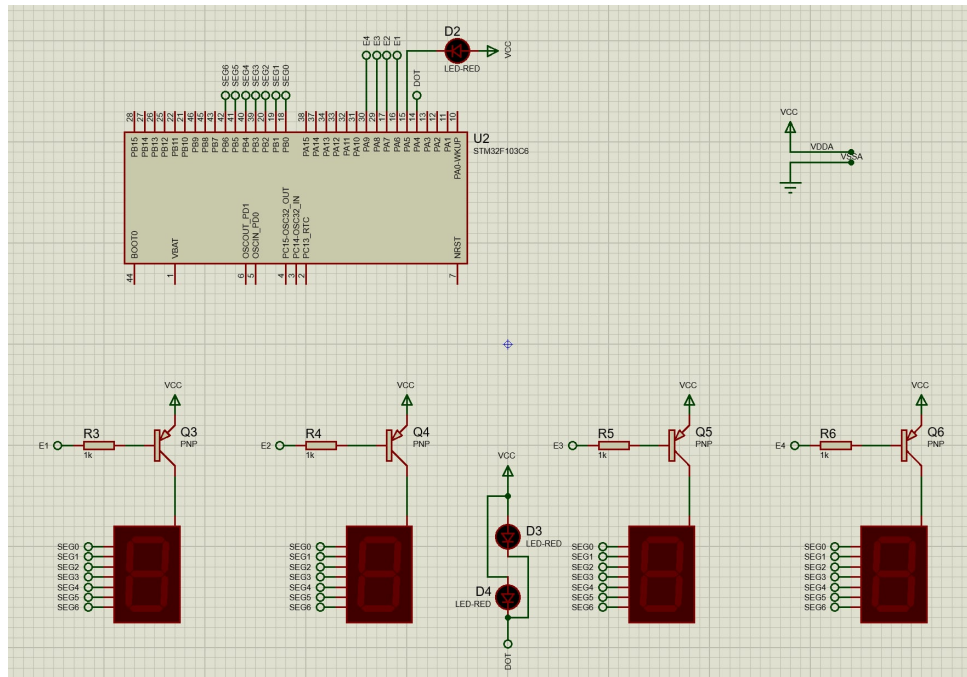


Figure 2: Proteus schematic for Exercise 2.

Source code for Exercise 2:

```
1 int ex2 = 1 ;
2 void run_ex2() {
3
4     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
5     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
6     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
7     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
8     if(ex2 == 1) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6,
9         GPIO_PIN_RESET);
10    else if(ex2 == 2) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7,
11        GPIO_PIN_RESET);
12    else if(ex2 == 3) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8,
13        GPIO_PIN_RESET);
```

```

11     else if(ex2 == 4) HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9,
12         GPIO_PIN_RESET);
13
14     display7SEG(ex2) ;
15     if(ex2 == 4) HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
16
17     ex2++ ;
18     if(ex2 == 5) ex2 = 1 ;
19 }

```

Code inside HAL-TIM-PeriodElapsedCallback function:

```

1  int clk = 0 ;
2  void HAL_TIM_PeriodElapsedCallback ( TIM_HandleTypeDef * htim )
3  {
4      clk++ ;
5      timer_run() ; clk = 0 ;
6      // if(clk == 51){ run_ex1() ; clk = 1 ;}
7      if(clk == 26){ run_ex2() ; clk = 1 ;}
8      // if(clk == 26){ update7SEG(index++); if(index >= MAX_LED)
9          index = 0 ; clk = 1 ;} // for ex 3, 4
10     // if(clk == 26){ run_ex5() ; update7SEG(index++); if(index
11         >= MAX_LED) index = 0 ; clk = 1 ;} // for ex 5
12 }

```

What is the frequency of the scanning process?

Similar to exercise 1, the switching time between two 7-segment LEDs is 0.5 seconds and there are four leds, so one complete scanning cycle takes 2 seconds. Therefore, resulting in a scanning frequency of 0,5 Hz.

3 Exercise 3

Source code for Exercise 3:

```
1  int led_buffer [4] = {4 , 3 , 2 , 1};
2  void update7SEG(int index) {
3      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
4      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
5      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
6      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
7      switch (index) {
8          case 0:
9              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
10             display7SEG(led_buffer[0]);
11             break;
12          case 1:
13              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
14              display7SEG(led_buffer[1]);
15              break;
16          case 2:
17              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
18              display7SEG(led_buffer[2]);
19              break;
20          case 3:
21              HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
22              display7SEG(led_buffer[3]);
23              HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
24              break;
25          default:
26              break;
27      }
28 }
```

Code inside HAL-TIM-PeriodElapsedCallback function:

```
1  int clk = 0 ;
2  void HAL_TIM_PeriodElapsedCallback ( TIM_HandleTypeDef * htim )
3  {
4      clk++ ;
5      timer_run() ; clk = 0 ;
6      // if(clk == 51){ run_ex1() ; clk = 1 ;}
7      // if(clk == 26){ run_ex2() ; clk = 1 ;}
8      if(clk == 26){ update7SEG(index++); if(index >= MAX_LED)
          index = 0 ; clk = 1 ;} // for ex 3, 4
9      // if(clk == 26){ run_ex5() ; update7SEG(index++); if(index
          >= MAX_LED) index = 0 ; clk = 1 ;} // for ex 5
10 }
```

4 Exercise 4

Code inside HAL-TIM-PeriodElapsedCallback function:

```
1  int clk = 0 ;
2  void HAL_TIM_PeriodElapsedCallback ( TIM_HandleTypeDef * htim )
3  {
4      clk++ ;
5      timer_run() ; clk = 0 ;
6      // if(clk == 51){ run_ex1() ; clk = 1 ;}
7      // if(clk == 26){ run_ex2() ; clk = 1 ;}
8      if(clk == 26){ update7SEG(index++); if(index >= MAX_LED)
          index = 0 ; clk = 1 ;} // for ex 3, 4
9      // if(clk == 26){ run_ex5() ; update7SEG(index++); if(index
          >= MAX_LED) index = 0 ; clk = 1 ;} // for ex 5
10 }
```

5 Exercise 5

Source code for Ex5:

```
1 void updateClockBuffer(int current_hour, int current_minute) {
2     // Calculate the tens and ones digits for the hour
3     led_buffer[0] = current_hour / 10;
4     led_buffer[1] = current_hour % 10;
5
6     // Calculate the tens and ones digits for the minute
7     led_buffer[2] = current_minute / 10;
8     led_buffer[3] = current_minute % 10;
9 }
10 int hour = 15 ; int minute = 20 ; int second = 36 ;
11 void run_ex5() {
12
13     second++;
14     if (second >= 60) {
15         second = 0;
16         minute++;
17     }
18     if (minute >= 60) {
19         minute = 0;
20         hour++;
21     }
22     if (hour >= 24) {
23         hour = 0;
24     }
25
26     updateClockBuffer(hour, minute);
27 }
```

Code inside HAL-TIM-PeriodElapsedCallback function:

```

1  int clk = 0 ;
2  void HAL_TIM_PeriodElapsedCallback ( TIM_HandleTypeDef * htim )
3  {
4      clk++ ;
5      timer_run() ; clk = 0 ;
6      // if(clk == 51){ run_ex1() ; clk = 1 ;}
7      // if(clk == 26){ run_ex2() ; clk = 1 ;}
8      // if(clk == 26){ update7SEG(index++); if(index >= MAX_LED)
          index = 0 ; clk = 1 ;} // for ex 3, 4
9      if(clk == 26){ run_ex5() ; update7SEG(index++); if(index >=
          MAX_LED) index = 0 ; clk = 1 ;} // for ex 5
10 }

```

6 Exercise 6

If in line 1 of the code above is miss, what happens after that and why?

If `setTimer0(1000)` is omitted, `timer0` counter remains at its initial value of 0. Therefore, `timer0` flag will not be set to 1, so the code inside the `if` statement will not execute. As a result, the LED will not toggle. The `setTimer0(2000)` inside the `if` statement also cannot be reached because the condition is never satisfied.

If in line 1 of the code above is changed to `setTimer0(1)`, what happens after that and why?

When `setTimer0(1)` is used, the specified duration is shorter than the 10 ms timer interrupt period. Therefore, `timer0` counter = $1/10 = 0$, so the software timer cannot reach the flag-triggering condition. As a result, `timer0` flag remains 0 and the code inside the `if` statement is not executed.

If in line 1 of the code above is changed to `setTimer0(10)`, what is changed compare to 2 first questions and why?

When `setTimer0(10)` is used, `timer0` counter is set to 1 because the timer interrupt period is 10 ms. Therefore, the first LED toggle occurs after 10 ms. After that, `setTimer0(2000)` resets the timer, so subsequent toggles occur every 2 seconds. Compared with `setTimer0(1000)`, only the initial delay is reduced from 1 second to 10 ms, while `setTimer0(1)` cannot trigger the timer because the specified duration is shorter than the 10 ms interrupt period.

7 Exercise 7 and Exercise 8

Software timmer function:

```
1  setTimer0 (100) ;
2      while (1)
3      {
4          int timer0_counter = 0;
5          int timer0_flag = 0;
6          int TIMER_CYCLE = 10;
7          void setTimer0 ( int duration ) {
8              timer0_counter = duration / TIMER_CYCLE ;
9              timer0_flag = 0;
10         }
11
12     int timer1_counter = 0;
13     int timer1_flag = 0;
14     void setTimer1 ( int duration ) {
15         timer1_counter = duration / TIMER_CYCLE ;
16         timer1_flag = 0;
17     }
18
19     void timer_run () {
20         if( timer0_counter > 0) {
```

```

20     timer0_counter-- ;
21     if( timer0_counter == 0) timer0_flag = 1;
22 }
23 if( timer1_counter > 0) {
24     timer1_counter-- ;
25     if( timer1_counter == 0) timer1_flag = 1;
26 }
27 }

```

Code inside while(1) loop:

```

1  setTimer0 (100) ;
2  while (1)
3  {
4      if( timer0_flag == 1) {
5          HAL_GPIO_TogglePin ( LED_RED_GPIO_Port , LED_RED_Pin )
6          ;
7          run_ex1() ;
8          run_ex2() ;
9          update7SEG(index++); if(index >= MAX_LED) index = 0 ;
10         run_ex5() ;
11         setTimer0 (250) ;
12     }
13 }

```

8 Exercise 9 and Exercise 10

Schematic from Proteus for Exercise:

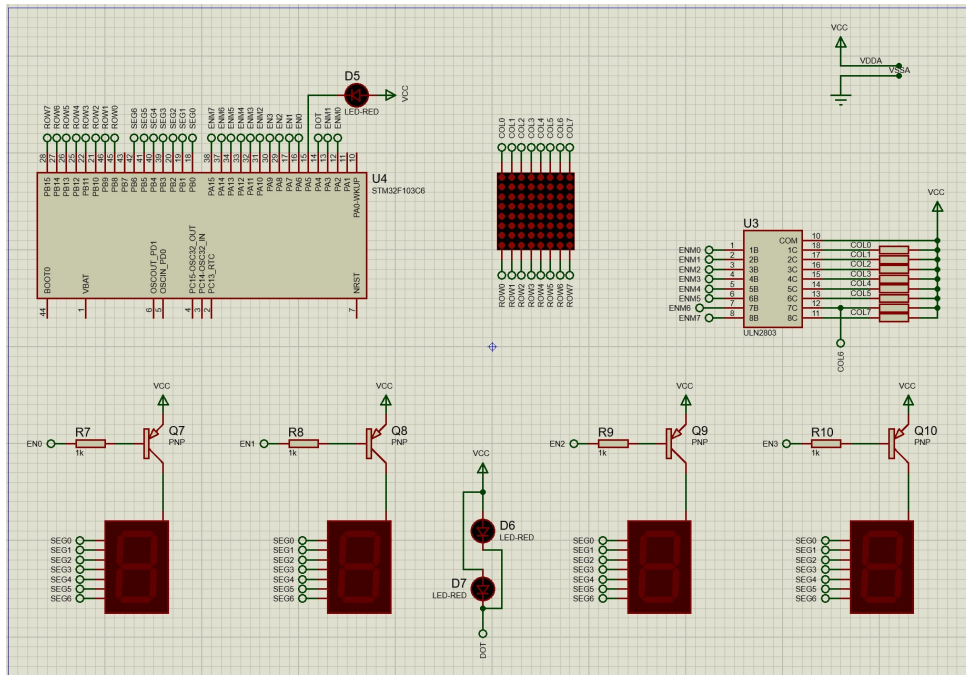


Figure 3: Proteus schematic for Exercise.

Source code for Exercise 9:

```

1 void displayRow(uint8_t row_data){
2     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, (row_data & 0x01) ?
3         GPIO_PIN_RESET : GPIO_PIN_SET);
4     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, (row_data & 0x02) ?
5         GPIO_PIN_RESET : GPIO_PIN_SET);
6     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, (row_data & 0x04) ?
7         GPIO_PIN_RESET : GPIO_PIN_SET);
8     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_11, (row_data & 0x08) ?
9         GPIO_PIN_RESET : GPIO_PIN_SET);
10    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, (row_data & 0x10) ?
11        GPIO_PIN_RESET : GPIO_PIN_SET);
12    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, (row_data & 0x20) ?
13        GPIO_PIN_RESET : GPIO_PIN_SET);
14    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, (row_data & 0x40) ?
15        GPIO_PIN_RESET : GPIO_PIN_SET);

```

```

9      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, (row_data & 0x80) ?
      GPIO_PIN_RESET : GPIO_PIN_SET);
10 }
11
12 uint8_t matrix_row [8] = {0x00 , 0xFC , 0x22 , 0x21 , 0x21 , 0x22
      , 0xFC , 0x00 };
13 void updateLEDMatrix(int index) {
14     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
15     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
16     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET);
17     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET);
18     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET);
19     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET);
20     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET);
21     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
22     switch (index) {
23         case 0:
24             shiftMatrixLeft() ;
25             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_RESET);
26             displayRow(matrix_row[index]) ;
27             break;
28         case 1:
29             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET);
30             displayRow(matrix_row[index]) ;
31             break;
32         case 2:
33             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET)
34             ;
35             displayRow(matrix_row[index]) ;
36             break;
37         case 3:

```

```

37         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET)
38         ;
39         displayRow(matrix_row[index]) ;
40         break;
41     case 4:
42         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET)
43         ;
44         displayRow(matrix_row[index]) ;
45         break;
46     case 5:
47         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET)
48         ;
49         displayRow(matrix_row[index]) ;
50         break;
51     case 6:
52         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET)
53         ;
54         displayRow(matrix_row[index]) ;
55         break;
56     case 7:
57         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET)
58         ;
59         displayRow(matrix_row[index]) ;
60         break;
61     default:
62         break;
63 }
64 }

```

Source code inside main():

```

1 int main(void)
2 {

```

```

3      /* Initialize all configured peripherals */
4      /* USER CODE BEGIN 2 */
5      HAL_TIM_Base_Start_IT (& htim2 ) ;
6      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET);
7      /* USER CODE END 2 */
8
9      /* Infinite loop */
10     /* USER CODE BEGIN WHILE */
11     setTimer0 (100) ;
12     setTimer1 (100) ;
13     while (1)
14     {
15         /* USER CODE END WHILE */
16
17         /* USER CODE BEGIN 3 */
18         if( timer0_flag == 1) {
19             HAL_GPIO_TogglePin ( LED_RED_GPIO_Port , LED_RED_Pin )
20             ;
21             run_ex1() ;
22             run_ex2() ;
23             update7SEG(index++); if(index >= MAX_LED) index = 0 ;
24             run_ex5() ;
25             setTimer0 (250) ;
26         }
27         if( timer1_flag == 1) {
28             if(index_matrix >= MAX_LED_MATRIX) index_matrix = 0 ;
29             updateLEDMatrix(index_matrix++) ;
30             setTimer1 (10) ;
31         }
32     }
33     /* USER CODE END 3 */
34 }

```

Explain:

For this exercise, I enable each column of matrix led serially across timer1 flag = 1. While the column is enable, I modify the row data matrix for describe suitable value for character "A".

Source code for Exercise 10:

```
1 void shiftMatrixLeft() {  
2     uint8_t tmp = matrix_row[0] ;  
3     for (int i = 0; i < 7; i++) {  
4         matrix_row[i] = matrix_row[i+1] ;  
5     }  
6     matrix_row[7] = tmp ;  
7 }
```

Explain:

For exercise 10, I call function shift left the matrix row buffer at the starting of the rotation describing the word "A".

GitHub Repository

My Github repository url: https://github.com/Minh-0405/BKU_Lab_MPU_MCU